

第7章 Flink | 开卷速查版

本章一句话：**Flink** 是一个原生流处理引擎，用来对**无界流**和**有界流**做**有状态计算**。它比 Storm 更强调状态、一致性和窗口语义；比 Spark Streaming 更偏真正的逐条流处理。🐼

常考题型： Flink vs Spark vs Storm；三种时间机制；**Watermark**；无状态/有状态算子；**Operator State / Keyed State**；状态后端；一致性检查点；保存点；Flink 如何统一流处理和批处理。

翻页关键词： Flink定位 → 对比Spark/Storm → 编程模型 → 架构 → 时间 → 窗口 → 水印 → 状态 → Operator State → Keyed State → State Backend → Checkpoint → Savepoint → 批流一体。

1. Flink 是什么

Flink 是一个分布式处理引擎，用于对**无界数据流**和**有界数据流**进行**有状态计算**。PPT里明确说 Flink 是“用于对无界和有界数据流进行有状态计算”的计算框架和分布式处理引擎。

无界数据流：数据一直来，没有明确结束，比如点击流、日志流、传感器数据流。**有界数据流：**有明确开始和结束，比如一个 HDFS 文件、一张静态表、一批历史日志。

考试写法： Flink 是面向有状态流处理的分布式计算框架，既可以处理无界数据流，也可以处理有界数据集。它原生支持事件时间、窗口、水印、有状态计算、一致性检查点和保存点，因此适合实时流处理和批流一体场景。

2. Flink 的优点

PPT里总结 Flink 的优点主要有：

同时支持高吞吐、低延迟、高性能。 Spark Streaming 主要是微批，吞吐和性能好，但低延迟不如原生流；Storm 低延迟较好，但高吞吐和状态一致性支持较弱。

支持事件时间和处理时间。 Flink 原生支持 **Event Time**，适合处理乱序数据。

提供 exactly-once 状态一致性保障。 也就是在故障恢复后，状态结果仍然像每条数据只被正确处理一次。

支持有状态流计算。 Flink 对状态有系统级管理，不只是用户自己在变量里维护。

支持灵活窗口和保存点。 可以做滚动窗口、滑动窗口、会话窗口等，也能通过 Savepoint 做升级、迁移、恢复。

考试写法： Flink 的优势在于原生流处理能力强，能够同时支持高吞吐、低延迟和高性能，并原生支持事件时间、水印、有状态计算和 exactly-once 状态一致性。同时，Flink 提供灵活窗口、轻量级分布式快照和保存点机制，适合可靠的实时流处理。

3. Flink vs Spark vs Storm

这个对比高频，直接背表。

对比点	Spark / Spark Streaming	Storm	Flink
本质	批处理引擎，流处理靠微批	实时流处理，逐条处理	原生流处理引擎
流处理方式	把流切成一小批一小批	数据来一条处理一条	数据来一条处理一条
延迟	微批延迟，通常高于真流	低延迟	低延迟
吞吐	高	相对弱一些	高
状态支持	有，但流式状态语义不如 Flink 原生	早期较弱，用户常自己维护	原生 Managed State
时间语义	支持但不是最核心	event time 支持弱	原生 Event Time + Watermark
一致性	依赖具体机制	早期多为 at-least-once	支持 exactly-once
批流融合思路	把流看成特殊批处理	主要做流	把批看成有限流

PPT里直接写：Spark 本质上是批处理引擎，通过 Spark Streaming 实现流处理；Flink 本质上是流处理引擎，通过有限数据流实现批处理。

考试写法：Spark 本质是批处理引擎，Spark Streaming 通过微批处理流数据；Storm 是低延迟流处理系统，但早期状态、窗口和 exactly-once 支持较弱；Flink 是原生流处理引擎，支持事件时间、水印、托管状态、一致性检查点和 exactly-once，并把批处理看成有限流处理。

4. Flink 编程模型：Dataflow

Flink 应用可以看成是一个 **Streaming Dataflow**，也就是一个 DAG。PPT里说：每个 Flink 应用可以映射为一个 streaming dataflow，由 stream 和 transformation operator 组成，operator 是节点，stream 是边，每个 dataflow 是一个 DAG，从 source 开始，到 sink 结束。

基本元素：

Stream：连续不断的数据记录组成的数据流。**Transformation / Operator：**作用在一个或多个 stream 上，输出一个或多个 stream。**Job：**一个 Flink 应用，对应一个 dataflow。**Source：**数据源，比如 Kafka、文件、socket。**Sink：**输出端，比如数据库、Kafka、文件系统。

图可以画成：

```

Source
|
v
Map / Filter Operator
|
v
KeyBy
|
v
Window / Aggregate Operator

```



```
graph TD; A[ ] --> B[Sink];
```

考试写法： Flink 程序可以表示为一个有向无环的数据流图。数据流 Stream 是边，Transformation 或 Operator 是节点；一个 Job 通常从一个或多个 Source 开始，经过 map、filter、keyBy、window、aggregation 等算子，最后输出到 Sink。

5. Operator、Subtask、Task、Operator Chain

Operator 是逻辑算子，比如 map、filter、window、aggregate。

Subtask 是 Operator 的一个并行实例。比如一个 map 算子并行度为 4，就有 4 个 map subtask。

Task 是运行时执行单元，可能包含一个或多个 subtask。

Operator Chain 是 Flink 把多个算子的 subtask 链接成一个执行链，减少线程切换和网络传输，提高吞吐、降低延迟。PPT里说 Flink 可以将多个 operator 的 subtask 链式拼接为一个 subtask，降低线程切换开销，增加吞吐并降低处理延时。

考试写法： Operator 是逻辑转换操作，Operator 的每个并行实例称为 Subtask。Task 是运行时执行单元，可能包含一个或多个 Subtask。Flink 可以通过 Operator Chain 将多个算子链式执行，从而减少线程切换和网络传输开销，提高吞吐并降低延迟。

6. Flink 架构：JobManager 和 TaskManager

Flink 是主从架构，核心是 **JobManager** 和 **TaskManager**。

JobManager： Master 进程，负责 Job 管理和资源协调。职责包括：接收 Job、生成 Execution Graph、调度 Task、管理 Checkpoint、失败恢复。

TaskManager： Worker 进程，具体执行 Task。每个 TaskManager 提供若干 **Task Slot**，负责执行 subtask、管理 slot、进行网络数据传输。PPT里把 JobManager 和 TaskManager 的职责分别列出。

Task Slot： TaskManager 中的一组固定资源，可以被一个或多个线程 / operator subtask 共享。不同 slot 中的 subtask 相互独立，不争夺资源。

考试写法： Flink 采用 JobManager / TaskManager 架构。JobManager 负责接收作业、生成执行图、调度任务、管理 checkpoint 和失败恢复；TaskManager 是执行任务的 worker 进程，提供 task slot，负责执行 subtask、管理资源和进行网络传输。

7. 三种时间机制

Flink 的时间机制必考。PPT里列出三种时间：**Event Time**、**Ingestion Time**、**Processing Time**。

Event Time，事件时间： 数据真实产生的时间。比如用户 11:02 点击按钮，这条点击事件的 event time 就是 11:02。

Ingestion Time，进入时间 / 提取时间： 数据进入 Flink 系统的时间。

Processing Time, 处理时间： 数据被某个 Task 实际处理时机器上的系统时间。

对比：

时间	含义	优点	缺点
Event Time	事件真实发生时间	结果更符合业务事实，能处理乱序	需要 watermark
Ingestion Time	进入 Flink 的时间	比 processing time 稳一点	仍不等于真实发生时间
Processing Time	被机器处理的时间	简单、延迟低	受网络延迟和机器调度影响，乱序时不准确

考试写法： Flink 支持事件时间、进入时间和处理时间。事件时间表示数据真实产生的时间，最符合业务语义；进入时间表示数据进入 Flink 的时间；处理时间表示数据被算子实际处理的机器时间。对于乱序流，通常使用 Event Time 配合 Watermark 来触发窗口计算。

8. 为什么需要 Watermark

流处理中数据可能**乱序到达**。PPT例子是：A 用户 11:02 操作，B 用户 11:03 操作，但 A 网络不稳定，导致系统先收到 B 的 11:03，再收到 A 的 11:02。

如果只按接收顺序处理，就会把 11:02 的事件当成“迟到”，窗口结果可能错。

所以需要 **Watermark 水印** 来告诉系统：**事件时间推进到哪里了，某个时间之前的数据基本都到了。**

9. Watermark 的定义

Watermark(t) 表示：系统认为**时间戳不大于 t 的数据都已经到来**，因此可以触发和销毁时间不超过 t 的窗口。

PPT里的定义就是：watermark(t) 表示所有时间戳不大于 t 的数据都已经到来，可以放心触发和销毁窗口。

注意几个词：

单调递增： Watermark 是时间进度标记，会向前推进。**处理乱序：** Watermark 允许数据晚到一段时间。**触发窗口：** 当 watermark 超过窗口结束时间，就可以计算该窗口。**太晚的数据：** 如果数据在窗口已经触发后才到，就可能成为 late data，需要额外处理或丢弃。

考试写法： Watermark 是 Flink 中处理事件时间乱序的机制。**Watermark(t)** 表示系统认为事件时间小于等于 t 的数据基本已经到达，因此可以触发结束时间不大于 t 的窗口计算并清理窗口状态。它通过允许一定乱序延迟，使 Event Time 计算结果更接近真实业务时间。

10. Watermark 例子

假设按事件时间做 1 分钟窗口：

```
窗口 [11:02, 11:03)
窗口 [11:03, 11:04)
```

事件到达顺序是：

```
事件 B: event_time = 11:03, 先到
事件 A: event_time = 11:02, 后到
```

如果没有 Watermark, 系统可能以为 11:02 已经过了, 提前输出窗口结果。

如果设置允许乱序 1 分钟, 系统可能生成：

```
watermark = 当前看到最大 event_time - 1分钟
```

看到 11:03 时, watermark 可能只是 11:02, 不会急着关闭 11:02 附近窗口; 这样 A 的 11:02 迟到事件还有机会被纳入正确窗口。

考试写法：Watermark 通过延迟推进事件时间进度来容忍乱序。例如当前看到的最大事件时间为 11:03, 若允许 1 分钟乱序, 则 watermark 可推进到 11:02。只有当 watermark 超过某窗口结束时间时, 该窗口才会被触发, 从而避免较早事件因网络延迟而被漏算。

11. 窗口 Window

流是无限的, 不能等“全部数据都来完”再算, 所以要切窗口。PPT里列出窗口类型：**滚动窗口**、**滑动窗口**、**会话窗口**、**全局窗口**；窗口定义依据有 **time-based** 和 **count-based**。

滚动窗口 Tumbling Window：固定大小, 窗口之间不重叠。例：每 5 分钟统计一次 PV, 窗口是 [0,5)、[5,10)。

滑动窗口 Sliding Window：固定大小, 但按滑动步长移动, 窗口可能重叠。例：每 1 分钟输出过去 5 分钟 PV。

会话窗口 Session Window：按用户活跃间隔切分, 超过一段时间没有事件就关闭会话。例：用户连续点击算一个 session。

全局窗口 Global Window：把整个数据集放到一个全局窗口里。Flink 用它解释批处理：有界数据可以看成是一个全局窗口上的流处理。

考试写法：窗口用于在无限数据流上划定有限计算范围。常见窗口包括滚动窗口、滑动窗口、会话窗口和全局窗口。窗口可以按时间定义, 也可以按记录数量定义。Flink 中批处理也可以理解为有界数据流上的全局窗口计算。

12. 状态 State：为什么需要

状态就是为了处理当前数据而保存的历史信息。流是无限的，很多计算不能只看当前一条数据。

PPT里把无状态和有状态这样区分：无状态算子一次只看一个事件，如 map、filter；有状态算子需要记住跨多个事件的信息，如 max、min、average、sum、median 等。状态场景包括去重、模式/趋势判断、窗口聚合、模型更新，窗口也可以看作一种状态。

无状态操作：

```
map
filter
单条记录字段转换
```

它们只看当前输入，不需要记住历史。

有状态操作：

```
count / sum / average
去重
窗口聚合
趋势判断
在线模型更新
流 join
```

这些都要保存历史或中间结果。

考试写法： State 是流处理中为后续计算保存的历史信息。无状态算子一次只处理单个事件，如 map、filter；有状态算子需要记住多个事件之间的信息，如计数、求和、去重、趋势判断、窗口聚合和在线模型更新。由于流是无限的，Flink 需要系统化地管理状态、容错和状态迁移。

13. 状态维护的挑战

Flink 为什么要专门支持状态管理？因为状态会带来三个难题：

状态可能很大。 数据流无界，状态可能超过内存。

故障时状态不能丢。 如果节点挂了，计数、窗口、去重集合都不能直接没了。

扩缩容时状态要迁移。 Rebalance 时，某些 key 可能换到其他 subtask，状态也要跟着迁移。

PPT里列出的挑战就是：处理超大 state、确保故障时 state 不丢失、rebalance 时 state 如何迁移。

考试写法： 状态管理的挑战在于：状态可能非常大，甚至超过内存；故障发生时状态不能丢失；任务并行度变化或 rebalance 时，状态需要正确迁移到新的 subtask。Flink 通过托管状态、状态后端和 checkpoint 机制解决这些问题。

14. Managed State 和 Raw State

Flink 中 state 一定与某个 operator 关联。为了让 Flink runtime 知道这个 operator 有状态，operator 需要注册它的 state。

PPT把 State 分成两类：

Managed State, 托管状态： 由 Flink 管理，Flink 负责存储、恢复和优化。考试重点。

Raw State, 原生状态： 开发者自己管理，Flink 不理解里面具体结构。一般不作为考试重点。

托管状态又分为：

```
Managed State
├── Operator State
└── Keyed State
```

考试写法： Flink 的状态分为 Managed State 和 Raw State。Managed State 由 Flink 运行时管理，负责状态的存储、恢复和优化，是常用方式；Raw State 由开发者自行管理。Managed State 又包括 Operator State 和 Keyed State。

15. Operator State

Operator State 被限定在某个 operator subtask 内。每个并行 subtask 都有自己的状态，其他 subtask 不能访问。

例子：Kafka Source 的每个并行实例记录自己读到的 partition offset；每个 subtask 都有自己的 offset 列表。

PPT里说 Operator State 被限定到一个 operator subtask，每个并行 subtask 都有自己的 state，无法被其他 subtask 访问；常见数据结构有 **List State** 和 **Broadcast State**。

常见类型：

List State： 状态以 list 表示，常用于 source offset 等。 **Broadcast State：** 当每个 task 的状态都相同时使用，比如广播规则配置。

考试写法： Operator State 绑定在某个 operator subtask 上，每个并行 subtask 拥有自己的状态，其他 subtask 无法访问。常见 Operator State 包括 List State 和 Broadcast State，适合保存与算子实例相关的状态，如 source offset 或广播规则。

16. Keyed State

Keyed State 用在 **keyBy** 之后。Flink 会为每个 key 维护一份状态，并保证相同 key 的记录被分区到维护该 key 状态的 operator subtask。

例子：

```
stream.keyBy(user_id).sum(amount)
```

这里每个 `user_id` 都有自己的 `sum` 状态。

PPT中说 Keyed State 适用于和 key 相关的 operator，Flink 对每个 key 维护一个 state 实例，并将所有同样 key 的记录分区到维护这些 key 的 state 的 operator subtask。

常见类型：

Value State： 每个 key 存一个值，比如当前 count。**List State：** 每个 key 存一个列表，比如某用户最近 N 次行为。**Map State：** 每个 key 存一个 key-value 映射，比如某用户不同商品的点击次数。

考试写法： Keyed State 是 keyBy 后使用的状态。Flink 会为每个 key 维护独立状态，并保证相同 key 的记录进入同一个负责该 key 的 subtask。常见 Keyed State 包括 Value State、List State 和 Map State，适合按用户、URL、设备等 key 做计数、去重、聚合和模式检测。

17. Operator State vs Keyed State

对比点	Operator State	Keyed State
绑定对象	operator subtask	key
使用前提	普通 operator 即可	必须 keyBy 后
状态粒度	每个 subtask 一份	每个 key 一份
谁能访问	当前 subtask	负责该 key 的 subtask
常见类型	List State、Broadcast State	Value State、List State、Map State
例子	Kafka Source offset、广播规则	每个用户计数、每个商品 PV

一句话记：**Operator State** 跟 **subtask** 走，**Keyed State** 跟 **key** 走。

18. 状态后端 State Backend

State Backend 是 Flink 管理状态的组件。PPT里说它负责两件事：**本地 state 管理**，以及为 **state 做 checkpoint 并存储到外部地址**。

它要解决：

状态存在哪里？
状态怎么读写？
checkpoint 时怎么持久化？
故障恢复时怎么加载？

PPT列出的管理方式：

方案1: JVM 堆内存。 状态存在 TaskManager JVM 堆中, 访问快, 但状态太大会占内存。

方案2: RocksDB。 状态序列化后存入 RocksDB。RocksDB 是持久化 Key-Value 存储引擎, 适合大状态, 但访问会有序列化和本地磁盘开销。

考试写法: State Backend 是 Flink 中负责存储、访问和维护状态的可插拔组件。它负责本地状态管理, 并在 checkpoint 时将状态持久化到外部存储。常见方式包括 JVM 堆内存和 RocksDB: 前者访问快但受内存限制, 后者适合大状态但有序列化和本地存储开销。

19. 一致性检查点 Checkpoint

Checkpoint 是 Flink 容错的核心。它会周期性地为所有有状态 subtask 保存一个一致的状态快照。

PPT中定义一致性检查点: 在所有 task 处理了一个相同输入后, 当前时间点每个 task 的 state 副本。朴素生成步骤是: 暂停输入流消费, 等待未处理数据处理完, 复制每个 task/subtask 的 state 到远端持久存储, 所有 task 完成后检查点完成, 再恢复消费输入流。

朴素理解流程:

1. 暂停输入流消费。
2. 等待当前已进入系统的数据处理完。
3. 把每个 `stateful subtask` 的状态复制到远端持久化存储。
4. 所有 `subtask` 状态都保存完成, `checkpoint` 完成。
5. 恢复输入流消费。

注意: PPT这里讲的是“朴素检查点生成步骤”, 真实 Flink 用分布式快照算法, 不一定真的长期暂停所有流; 考试按 PPT 写朴素理解即可。

考试写法: Flink 通过一致性 checkpoint 实现容错。Checkpoint 可以理解为在所有 subtask 处理到同一个逻辑输入位置时, 对每个 `stateful subtask` 的状态做一次快照, 并将其保存到远端持久化存储。故障后, Flink 可以从最近一次 checkpoint 恢复所有状态并继续处理。

20. 从 Checkpoint 恢复

PPT给出的恢复步骤:

1. 重启整个 `application`。
2. 将所有 `stateful subtasks` 的状态重置为最近的 `checkpoint`。
3. 恢复所有 `subtasks` 的处理。
4. 配合 `Kafka` 等可重放数据源, 实现 `exactly-once`。

为什么能 `exactly-once`? 因为恢复时, 不只是恢复 Flink 内部 state, 还要让数据源回到 checkpoint 对应的位置。比如 `Kafka offset` 也被保存下来, 恢复后从对应 `offset` 继续消费。

考试写法: 从 checkpoint 恢复时, Flink 会重启应用, 将所有有状态 subtask 的状态恢复到最近一次 checkpoint, 然后从相应输入位置继续处理。若输入源如 `Kafka` 支持 `offset` 回放, 并且 sink 也支持一致性提

交，就可以实现端到端 exactly-once 语义。

21. Checkpoint 例子：奇偶求和

PPT例子是 source 输出递增数字，按照奇偶数分成两个 subtask，每个 subtask 对输入求和。

可以这样理解：

```
Source: 1,2,3,4,5,6,...
```

keyBy 奇偶：

odd subtask: 1,3,5,... state = odd_sum

even subtask: 2,4,6,... state = even_sum

某时刻 checkpoint：

```
odd_sum = 1 + 3 + 5 = 9
```

```
even_sum = 2 + 4 + 6 = 12
```

如果之后系统故障，恢复时：

```
odd_sum 恢复为 9
```

```
even_sum 恢复为 12
```

```
source 从 checkpoint 对应位置继续发数据
```

这样不会从 0 开始，也不会把已经算过的数据重复算进状态。

22. Savepoint 保存点

Savepoint 和 Checkpoint 原理类似，都是保存应用状态。但用途不一样。

PPT里说：保存点和检查点用相同算法创建；保存点 = 检查点 + 一些额外元数据；需要用户或外部调度器明确触发；Flink 不会自动清理保存点。

Checkpoint：自动、周期性、用于故障恢复，Flink 可能自动清理。**Savepoint**：手动触发、用于运维、升级、迁移、版本管理，Flink 不自动清理。

考试写法：Savepoint 与 Checkpoint 使用类似的快照机制，但 Savepoint 由用户或外部调度器显式触发，并带有额外元数据，Flink 不会自动清理。Checkpoint 主要用于自动容错恢复；Savepoint 主要用于应用升级、迁移、扩缩容、A/B 测试、暂停恢复和版本管理。

23. Savepoint 有什么用

PPT里列出 Savepoint 可以用于：

- 从保存点启动同一个应用。比如暂停后恢复。
- 从保存点启动兼容的新应用。比如修 bug 后重新处理部分数据。
- A/B 测试或不同业务逻辑测试。
- 改变并行度。例如扩容或缩容应用。
- 迁移集群或升级 Flink 版本。
- 释放资源。先做 savepoint，停掉应用，之后再恢复。
- 归档状态做版本管理。

考试写法： Savepoint 可用于应用升级、修复逻辑 bug、A/B 测试、调整并行度、迁移集群、升级 Flink 版本、暂停并恢复应用，以及归档应用状态。它要求新的应用与保存点状态结构兼容。

24. Checkpoint vs Savepoint

对比点	Checkpoint	Savepoint
触发方式	自动周期性触发	用户/外部调度器手动触发
主要用途	故障容错恢复	运维、升级、迁移、版本管理
是否自动清理	通常由系统管理	Flink 不自动清理
是否有额外元数据	较少	有额外元数据
使用场景	节点挂了恢复	改代码、改并行度、迁集群
口诀	自动容错	人工运维

一句话记：**Checkpoint 管故障，Savepoint 管运维。**

25. Flink 批处理思路：把批看成有限流

这是 Flink 和 Spark 对比的重点。

PPT里说：

Spark 的批流融合思路： 将流处理作为特殊的批处理，本质是批处理引擎。**Flink 的批流融合思路：** 将批处理作为特殊的流处理，和 Spark 正好相反。

Flink 看法：

无限数据流：没有结束

有限数据流：无限数据流的特例，有开始、有结束

批处理就是处理一个有限流。Flink 还可以用 **全局窗口** 理解批处理：把整个数据集的时间周期看作一个全局窗口，在这个窗口里处理完所有数据。

例子：

流处理：

窗口 [1,3], [4,6], [7,9] 分别输出

批处理：

整个 [1,9] 看成一个全局窗口，一次性处理

考试写法： Flink 的批流一体思想是把批处理看成有限数据流上的特殊流处理。有界数据流是无界数据流的特例，它从某个时间点开始并在某个时间点结束。批处理可以看成对整个数据集设置一个全局窗口的流处理。与之相反，Spark 本质是批处理引擎，把流处理实现为微批处理。

26. Flink 链路设计题模板

如果题目给一个流，让你设计 Flink 程序或伪代码，按这个模板写：

1. Source: 从 Kafka / 文件 / socket 读入数据。
2. map / flatMap: 解析数据。
3. filter: 过滤无效数据。
4. keyBy: 按需要维护状态或聚合的字段分区。
5. window: 如果是时间范围统计，定义窗口。
6. aggregate / process: 聚合、去重、趋势判断等。
7. sink: 输出到数据库 / Kafka / 文件 / 告警系统。

例：按 URL 统计每 5 分钟 PV

```
KafkaSource
-> map(parseLog)
-> filter(valid)
-> keyBy(url)
-> window(TumblingEventTimeWindow 5min)
-> sum(count)
-> Sink
```

解释：

map/filter 是无状态。**keyBy(url)** 保证同一个 URL 到同一个 keyed state。**window + sum** 需要状态。**watermark** 用于事件时间乱序。

考试写法：

Flink 流处理程序应先从 **Source** 读取数据，再通过 **map/filter** 做无状态清洗；如果需要按某个字段统计或维护状态，应使用 **keyBy** 将相同 **key** 的数据分到同一状态实例；若是时间范围统计，则定义 **window** 并配合 **watermark** 处理乱序；最后通过 **aggregate/process** 得到结果并写入 **Sink**。

本章高频简答模板

1. Flink 是什么？ Flink 是面向有界和无界数据流的分布式有状态计算引擎。它原生支持流处理、事件时间、水印、窗口、有状态计算、一致性 checkpoint 和 savepoint，适合可靠的实时流处理和批流一体场景。

2. Flink 和 Spark Streaming 的区别？ Spark 本质是批处理引擎，Spark Streaming 将流数据切分成微批处理；Flink 本质是流处理引擎，将批处理看成有限流处理。Flink 原生支持事件时间、水印、托管状态和 exactly-once，更适合低延迟有状态流处理。

3. Flink 和 Storm 的区别？ Storm 是低延迟实时流处理系统，但早期状态、窗口、event time 和 exactly-once 支持较弱；Flink 同样支持低延迟流处理，同时提供更完善的状态管理、窗口、水印、一致性 checkpoint 和批流一体能力。

4. Watermark 是什么？ Watermark 是处理事件时间乱序的机制。`Watermark(t)` 表示系统认为时间戳不大于 t 的数据基本已经到达，可以触发结束时间不大于 t 的窗口计算并清理窗口状态。

5. 什么是状态？为什么 Flink 需要状态管理？ 状态是流处理中为了处理后续数据而保存的历史信息，如计数值、去重集合、窗口内容和模型参数。由于流是无限的，状态可能很大，并且故障恢复和扩缩容时必须保持一致，所以 Flink 需要托管状态、状态后端和 checkpoint 机制。

6. Operator State 和 Keyed State 的区别？ Operator State 绑定到某个 operator subtask，每个并行 subtask 有自己的状态；Keyed State 绑定到 key，必须在 keyBy 后使用，Flink 为每个 key 维护独立状态，并保证相同 key 的数据进入同一个负责该 key 的 subtask。

7. State Backend 是什么？ State Backend 是 Flink 中管理状态存储、访问和 checkpoint 持久化的组件。它负责本地状态管理，并在 checkpoint 时把状态保存到外部存储。常见方式有 JVM 堆内存和 RocksDB。

8. Checkpoint 是什么？ Checkpoint 是 Flink 的一致性快照机制，用于容错恢复。它在所有 stateful subtask 处理到同一逻辑输入位置时保存状态快照；故障后，应用可以从最近 checkpoint 恢复状态并继续处理。

9. Savepoint 和 Checkpoint 有什么区别？ Checkpoint 自动周期性生成，主要用于故障容错；Savepoint 由用户显式触发，用于升级、迁移、调整并行度、暂停恢复、A/B 测试和版本管理。Checkpoint 管故障，Savepoint 管运维。

10. Flink 如何统一批处理和流处理？ Flink 将批处理看成有限数据流上的流处理。有界数据是无界数据的特例，可以通过全局窗口处理整个数据集。因此 Flink 的批流一体思路是“批是特殊的流”，而 Spark 的思路是“流是特殊的批”。

本章易混点

Event Time 不是处理时间。 Event Time 是事件真实发生时间；Processing Time 是机器处理它的时间。

Watermark 不是窗口。 Watermark 是触发窗口的时间进度标记；窗口是数据分组计算范围。

有状态不等于一定用 Keyed State。 如果状态跟 subtask 绑定，用 Operator State；如果状态跟 key 绑定，用 Keyed State。

keyBy 很重要。 按用户、URL、设备统计时，必须 keyBy 对应字段，否则相同 key 的状态会分散。

Checkpoint 和 Savepoint 不要混。 Checkpoint 是自动容错；Savepoint 是手动运维。

Flink 批流一体方向和 Spark 相反。 Spark 是批处理引擎做微批流；Flink 是流处理引擎处理有限流。

本章看到题目怎么写

看到“**Flink vs Spark**”：写 Spark 微批、批处理本质；Flink 原生流，把批当有限流。

看到“**Flink vs Storm**”：写 Storm 低延迟但状态/窗口/exactly-once 弱；Flink 状态、水印、checkpoint 完整。

看到“**三种时间**”：写 Event Time、Ingestion Time、Processing Time，乱序场景优先 Event Time + Watermark。

看到“**Watermark**”：写 `watermark(t)` 表示 t 之前数据基本到齐，可以触发窗口。

看到“**状态题**”：先写无状态 map/filter，再写有状态 count/sum/去重/窗口/趋势；最后写 Flink 用 Managed State 管理。

看到“**Operator State / Keyed State**”：写 Operator State 跟 subtask，Keyed State 跟 key；keyBy 后用 Keyed State。

看到“**容错题**”：写 checkpoint 保存所有 stateful subtask 状态，恢复时从最近 checkpoint 重启；配合 Kafka 可 exactly-once。

看到“**保存点题**”：写 savepoint 与 checkpoint 类似，但手动触发，用于升级、迁移、改并行度、暂停恢复。